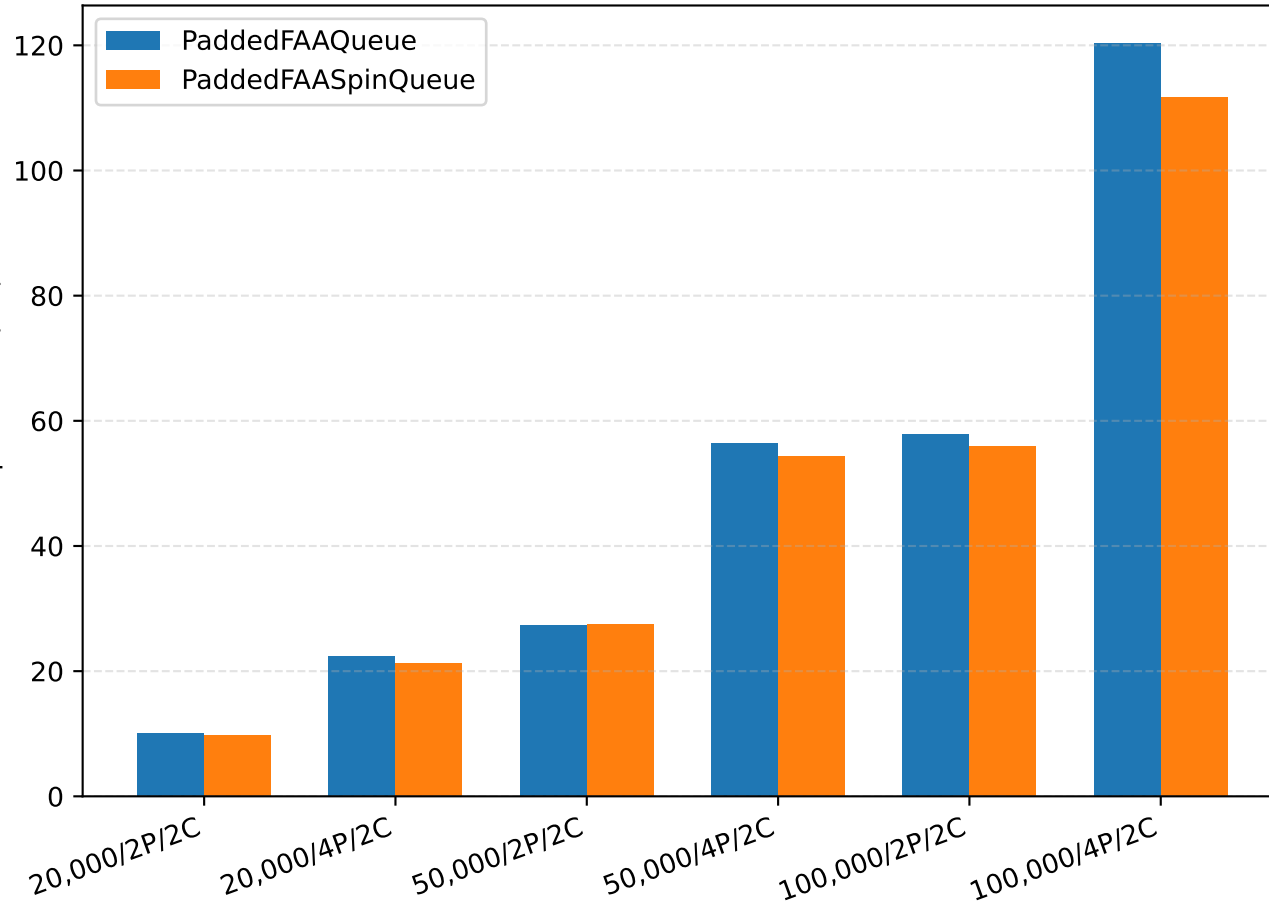
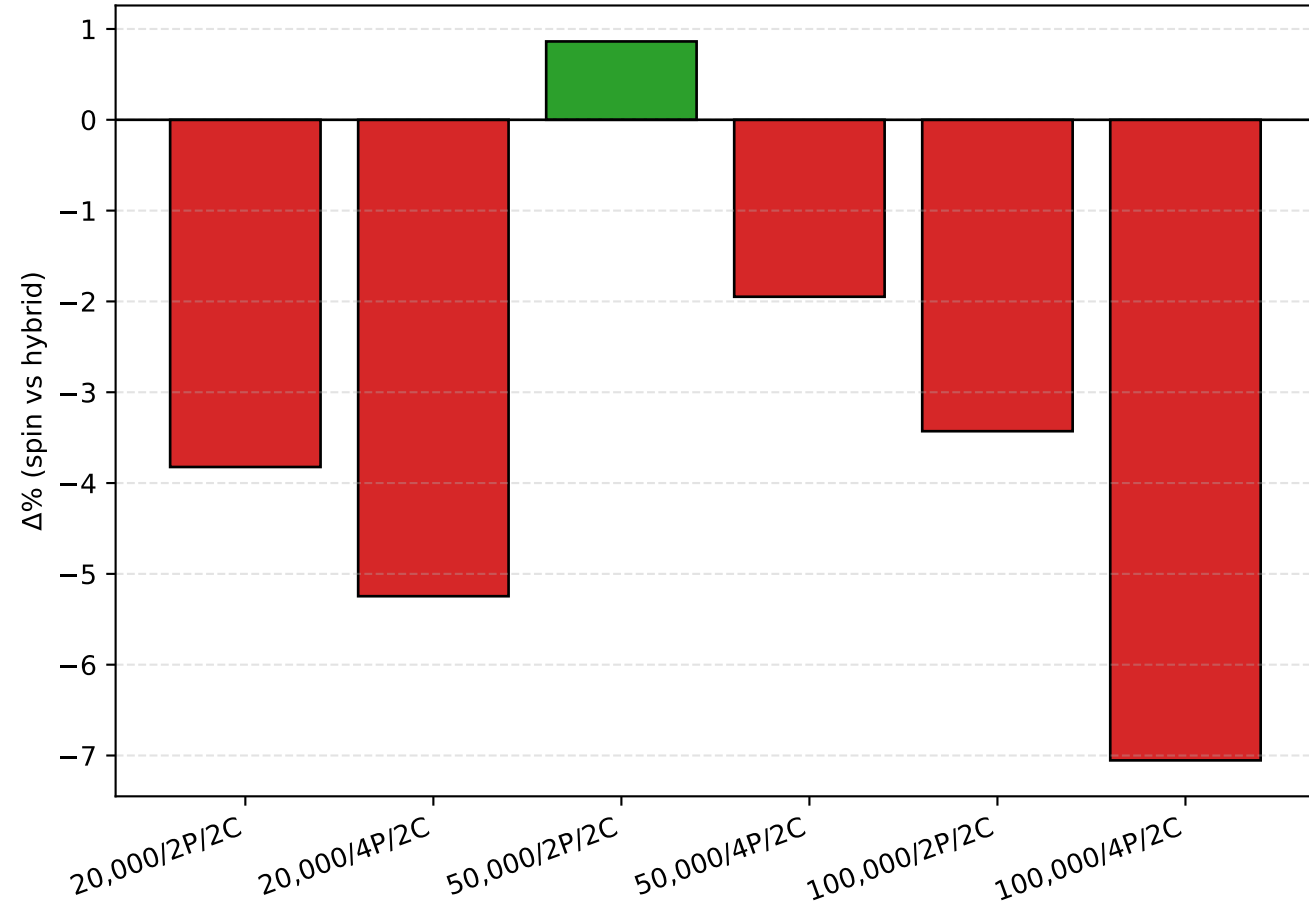


PaddedFAAQueue vs PaddedFAASpinQueue: costo di sleep(0.0)

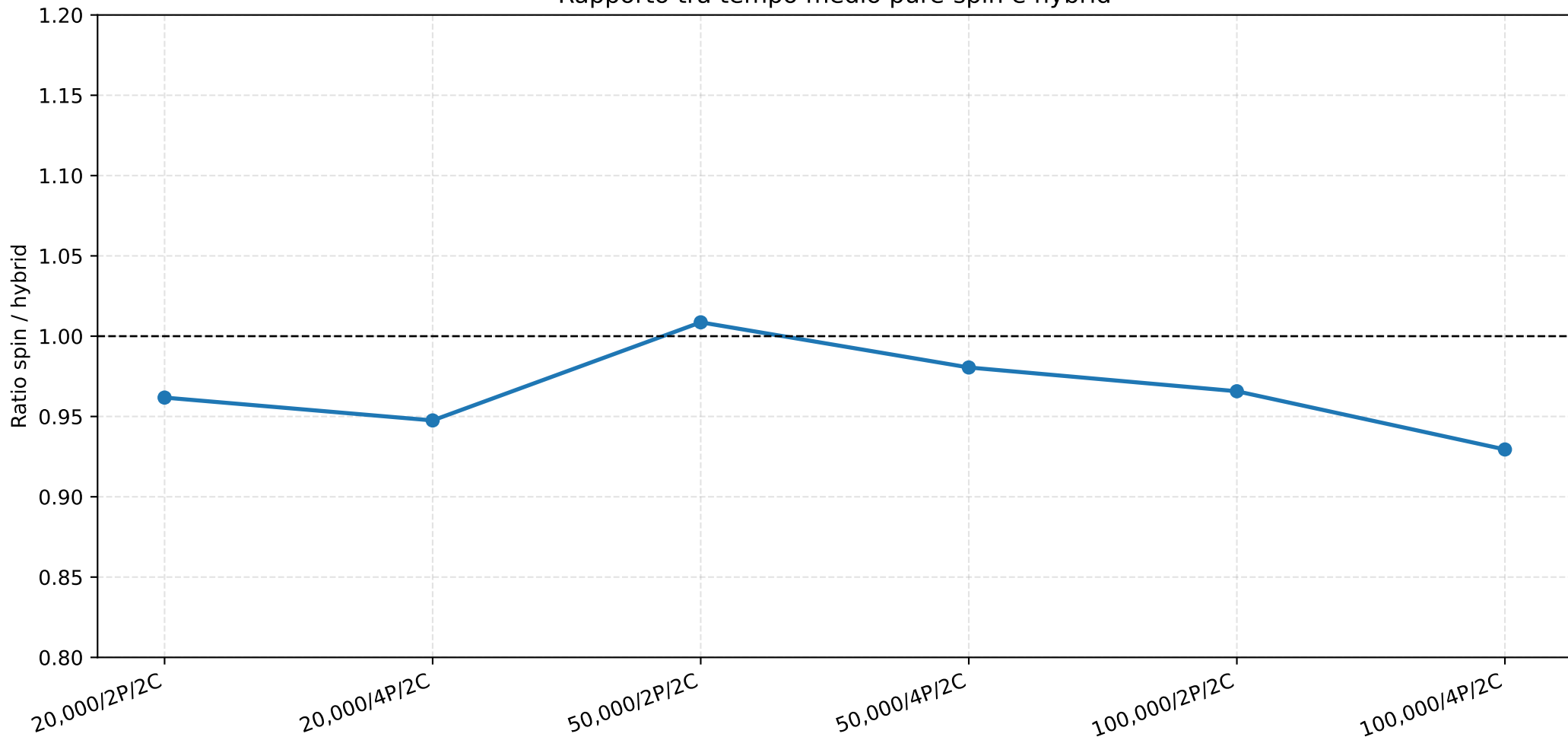
Confronto tempi medi



Variazione percentuale del pure-spin



Rapporto tra tempo medio pure-spin e hybrid



Sintesi delle medie per configurazione

Config	Hybrid ms	Spin ms	Ratio S/H	$\Delta\%$
20,000/2P/2C	10.13	9.76	0.962	-3.82
20,000/4P/2C	22.45	21.26	0.948	-5.25
50,000/2P/2C	27.43	27.55	1.009	0.86
50,000/4P/2C	56.40	54.32	0.981	-1.95
100,000/2P/2C	57.91	55.90	0.966	-3.43
100,000/4P/2C	120.35	111.70	0.929	-7.05

Interpretazione dei risultati

Il confronto è stato costruito per isolare il costo di `sleep(0.0)` senza cambiare il resto del runtime.

Le due code condividono lo stesso layout di slot, la stessa capacità, gli stessi ticket FAA e lo stesso ordine di scheduling dei worker. L'unica differenza è la policy di attesa: la variante ibrida fa spin fino a 1024 iterazioni e poi cede la CPU, mentre la pure-spin resta in busy wait continuo.

In pratica, il benchmark misura quanto il yield esplicito stia costando in throughput e latenza sotto contenimento.

I dati mostrano un comportamento non uniforme: per 20k e 100k messaggi con più producer, il pure-spin resta più veloce, mentre per 50k il rapporto si avvicina a 1 e in alcuni casi la variante ibrida può pareggiare o superare il pure-spin.

Questo indica che `sleep(0.0)` non è necessariamente gratis ma il suo beneficio dipende dalle condizioni di contesa e dal regime del sistema: in carico medio-alto il pure-spin paga meno overhead di schedulazione; in carico più basso il beneficio del yield può diventare marginale o addirittura controproducente.

La conclusione più prudente è che `sleep(0.0)` è una policy di backoff utile per contenere il busy-wait, ma il suo costo va misurato empiricamente e non assunto a priori.

Nota: i grafici sono ottenuti dai dati raccolti con la stessa pipeline, stesso workload e stesso seed di scheduling per ogni esecuzione.